LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Muhammad Kurniawan Pasya NIM. 2510817210026

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack danQueue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya
NIM : 2510817210026

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 6
B. Output Program 13
C. Pembahasan 14
SOAL 2 16
A. Source Code 17
B. Output Program 20
C. Pembahasan 20
SOAL 3 26
A. Source Code 26
B. Output Program 34
C. Pembahasan 35
SOAL 4 37
A. Source Code 38
B. Output Program 41
C. Pembahasan 41
TAUTAN GIT 48

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 13
Gambar 2. Screenshot Hasil Jawaban Soal 2 20
Gambar 3. Screenshot Hasil Jawaban Soal 3 34
Gambar 4. Screenshot Hasil Jawaban Soal 4 41

DAFTAR TABEL

Tabel 1 Source test_stack.cpp 6
Tabel 2 Source Code stack.cpp..... 11
Tabel 3 Source Code stack.h 12
Tabel 4 Source Code prob1.cpp 17
Tabel 5 SourceCode stack.cpp..... 18
Tabel 6 Source Code stack.h 20
Tabel 7 Source Code test_queue.cpp..... 26
Tabel 8 Source Code queue.cpp 31
Tabel 9 Source Code queue.h..... 33
Tabel 10 Source Code prob2.cpp 38
Tabel 11 Source Code queue.cpp 39
Tabel 12 Source Code queue.h..... 41

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source test_stack.cpp

```
// tests/test_stack.cpp
// Exhaustive unit tests for Stack ADT (stack.h / stack.cpp)
// exit 0 = all pass, exit 1 = any failure

#include "../stack.h"
#include <iostream>
#include <string>
#include <stdexcept>

static int g_passed = 0, g_failed = 0;

void test(const std::string& name, bool cond) {
    if (cond) { std::cout << "[PASS] " << name << "\n"; ++g_passed; }
    else      { std::cout << "[FAIL] " << name << "\n"; ++g_failed; }
}

template<typename Fn>
bool throws(Fn fn) {
    try { fn(); return false; }
    catch (...) { return true; }
}

int main() {
```

```
    std::cout << "=== Stack Unit Tests ===\n\n";

    // §1. Initialization
    Stack s;
    init(&s);
    test("§1.1 init: isEmpty",         isEmpty(&s));
    test("§1.2 init: !isFull",        !isFull(&s));
    test("§1.3  init:  peek  throws",         throws([&]{
peek(&s); }));
    test("§1.4  init:  pop  throws",          throws([&]{
pop(&s); }));

    // §2. Single push/pop
    push(&s, 42);
    test("§2.1 push(42): !isEmpty",   !isEmpty(&s));
    test("§2.2 push(42): !isFull",    !isFull(&s));
    test("§2.3 push(42): peek==42",    peek(&s) == 42);
    pop(&s);
    test("§2.4 after pop: isEmpty",    isEmpty(&s));
    test("§2.5  after  pop:  peek  throws",throws([&]{
peek(&s); }));
    test("§2.6  after  pop:  pop  throws",  throws([&]{
pop(&s); }));

    // §3. LIFO — 3 elements
    push(&s, 10); push(&s, 20); push(&s, 30);
    test("§3.1 peek==30", peek(&s)==30); pop(&s);
    test("§3.2 peek==20", peek(&s)==20); pop(&s);
    test("§3.3 peek==10", peek(&s)==10); pop(&s);
    test("§3.4 empty after 3 pops", isEmpty(&s));

    // §4. LIFO — 10 elements sequential
    for (int i = 1; i <= 10; ++i) push(&s, i * 100);
    for (int i = 10; i >= 1; --i) {
        test("§4  LIFO  peek=="  +  std::to_string(i*100),
peek(&s) == i*100);
        pop(&s);
    }
    test("§4.end empty", isEmpty(&s));

    // §5. Boundary values
    push(&s, -1000); push(&s, 0); push(&s, 1000);
    test("§5.1 peek==1000",  peek(&s)==1000);  pop(&s);
    test("§5.2 peek==0",     peek(&s)==0);     pop(&s);
    test("§5.3 peek==-1000", peek(&s)==-1000); pop(&s);

    // §6. Duplicate values
```

```
    push(&s, 7); push(&s, 7); push(&s, 7);
    test("§6.1 dup top1==7", peek(&s)==7); pop(&s);
    test("§6.2 dup top2==7", peek(&s)==7); pop(&s);
    test("§6.3 dup top3==7", peek(&s)==7); pop(&s);
    test("§6.4 empty after dups", isEmpty(&s));

    // §7. Interleaved push/pop
    push(&s, 1);
    test("§7.1 peek==1", peek(&s)==1);
    push(&s, 2);
    test("§7.2 peek==2", peek(&s)==2);
    pop(&s);
    test("§7.3 back to peek==1", peek(&s)==1);
    push(&s, 3);
    test("§7.4 peek==3", peek(&s)==3);
    pop(&s); pop(&s);
    test("§7.5 empty", isEmpty(&s));

    // §8. isFull at MAX capacity
    for (int i = 0; i < MAX; ++i) push(&s, i);
    test("§8.1 isFull",           isFull(&s));
    test("§8.2 !isEmpty",        !isEmpty(&s));
    test("§8.3 push-full throws", throws([&]{ push(&s,
999); }));
    test("§8.4 peek is last",     peek(&s) == MAX - 1);

    // §9. Pop all from full, verify each value
    for (int i = MAX - 1; i >= 0; --i) {
        test("§9 drain peek==" + std::to_string(i),
peek(&s) == i);
        pop(&s);
    }
    test("§9.end empty after full drain", isEmpty(&s));

    // §10. Re-init resets
    push(&s, 55); push(&s, 66);
    init(&s);
    test("§10.1 isEmpty after re-init",
isEmpty(&s));
    test("§10.2 peek throws after re-init",
throws([&]{ peek(&s); }));
    test("§10.3 pop throws after re-init", throws([&]{
pop(&s); }));
    push(&s, 99);
    test("§10.4 push after re-init peek==99",
peek(&s)==99);
    pop(&s);
```

```
    test("§10.5   empty   after   pop   post-reinit",
isEmpty(&s));

    // §11. Multiple independent stacks
    Stack s2;
    init(&s2);
    push(&s, 100); push(&s2, 200);
    test("§11.1 s peek==100",  peek(&s)==100);
    test("§11.2 s2 peek==200", peek(&s2)==200);
    push(&s, 101); push(&s2, 201);
    test("§11.3 s peek==101",  peek(&s)==101);
    test("§11.4 s2 peek==201", peek(&s2)==201);
    pop(&s); pop(&s);
    pop(&s2); pop(&s2);
    test("§11.5 s empty",  isEmpty(&s));
    test("§11.6 s2 empty", isEmpty(&s2));

    // §12. isFull/isEmpty boundary dance
    for (int i = 0; i < MAX; ++i) push(&s, i);
    test("§12.1 isFull",               isFull(&s));
    pop(&s);
    test("§12.2 !isFull after 1 pop",  !isFull(&s));
    push(&s, 9999);
    test("§12.3 isFull after re-fill", isFull(&s));
    test("§12.4  push  still  throws",       throws([&]{
push(&s, 1); }));

    // §13. Drain full stack, refill with negatives
    while (!isEmpty(&s)) pop(&s);
    test("§13.1 drained",   isEmpty(&s));
    for (int i = 0; i < MAX; ++i) push(&s, -i);
    test("§13.2 full again",          isFull(&s));
    test("§13.3 top is -(MAX-1)",     peek(&s) == -(MAX-
1));
    int expected_neg = -(MAX-1);
    pop(&s);
    test("§13.4  after  1  pop:  -(MAX-2)",  peek(&s)  ==
expected_neg + 1);

    // §14. Negative values LIFO order
    while (!isEmpty(&s)) pop(&s);
    push(&s, -1); push(&s, -2); push(&s, -3);
    test("§14.1 peek==-3", peek(&s)==-3); pop(&s);
    test("§14.2 peek==-2", peek(&s)==-2); pop(&s);
    test("§14.3 peek==-1", peek(&s)==-1); pop(&s);

    // §15. Zero-only stack
```

```
    push(&s, 0); push(&s, 0); push(&s, 0);
    test("§15.1 peek==0 #1", peek(&s)==0); pop(&s);
    test("§15.2 peek==0 #2", peek(&s)==0); pop(&s);
    test("§15.3 peek==0 #3", peek(&s)==0); pop(&s);
    test("§15.4 empty",      isEmpty(&s));

    // §16. Mixed positive/negative interleaved
    push(&s,  -500);  push(&s,  500);  push(&s,  -250);
push(&s, 250);
    test("§16.1 peek==250",  peek(&s)==250);  pop(&s);
    test("§16.2 peek==-250", peek(&s)==-250); pop(&s);
    test("§16.3 peek==500",  peek(&s)==500);  pop(&s);
    test("§16.4 peek==-500", peek(&s)==-500); pop(&s);

    //  §17.  Exception  is  actually  catchable  (not
crash/abort)
    bool  caught_pop  =  false,  caught_peek  =  false,
caught_full = false;
    try { pop(&s); }  catch (...) { caught_pop = true; }
    test("§17.1 pop-empty catchable", caught_pop);

    try { peek(&s); } catch (...) { caught_peek = true;
}
    test("§17.2 peek-empty catchable", caught_peek);

    for (int i = 0; i < MAX; ++i) push(&s, i);
    try { push(&s, 1); } catch (...) { caught_full = true;
}
    test("§17.3 push-full catchable", caught_full);

    // §18. Multiple pops exactly to empty, then one more
throws
    while (!isEmpty(&s)) pop(&s);
    push(&s, 1); push(&s, 2);
    pop(&s); pop(&s);
    test("§18.1 empty exactly", isEmpty(&s));
    test("§18.2  one-more  pop  throws",  throws([&]{
pop(&s); }));

    // §19. Peek does not modify stack
    push(&s, 77);
    test("§19.1 peek==77",       peek(&s)==77);
    test("§19.2 peek==77 again", peek(&s)==77);
    test("§19.3 !isEmpty",      !isEmpty(&s));
    pop(&s);
    test("§19.4 empty after pop", isEmpty(&s));
```

```
    // §20. Large value stress
    push(&s, 999999); push(&s, -999999);
    test("§20.1   peek==-999999",   peek(&s)==-999999);
pop(&s);
    test("§20.2   peek==999999",      peek(&s)==999999);
pop(&s);

    // Summary
    std::cout << "\n=== " << g_passed << " / " <<
(g_passed + g_failed)
              << " tests passed ===\n";
    std::cout << "\nTekan Enter buat nutup...";
    std::cin.get();
    return (g_failed == 0) ? 0 : 1;
}
```

Tabel 2 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Stack Overflow");
    }
    if (isEmpty(s)) {
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }
    *(s->top) = value;
}
```

```

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack Underflow");
    }
    if (s->top == &(s->data[0])) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack is Empty");
    }
    return *(s->top);
}
```

Tabel 3 Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

B. Output Program

```
D:\Penyimpanan Utama\Docu

[PASS] ┬°10.1 isEmpty after re-init
[PASS] ┬°10.2 peek throws after re-init
[PASS] ┬°10.3 pop throws after re-init
[PASS] ┬°10.4 push after re-init peek==99
[PASS] ┬°10.5 empty after pop post-reinit
[PASS] ┬°11.1 s peek==100
[PASS] ┬°11.2 s2 peek==200
[PASS] ┬°11.3 s peek==101
[PASS] ┬°11.4 s2 peek==201
[PASS] ┬°11.5 s empty
[PASS] ┬°11.6 s2 empty
[PASS] ┬°12.1 isFull
[PASS] ┬°12.2 !isFull after 1 pop
[PASS] ┬°12.3 isFull after re-fill
[PASS] ┬°12.4 push still throws
[PASS] ┬°13.1 drained
[PASS] ┬°13.2 full again
[PASS] ┬°13.3 top is -(MAX-1)
[PASS] ┬°13.4 after 1 pop: -(MAX-2)
[PASS] ┬°14.1 peek==-3
[PASS] ┬°14.2 peek==-2
[PASS] ┬°14.3 peek==-1
[PASS] ┬°15.1 peek==0 #1
[PASS] ┬°15.2 peek==0 #2
[PASS] ┬°15.3 peek==0 #3
[PASS] ┬°15.4 empty
[PASS] ┬°16.1 peek==250
[PASS] ┬°16.2 peek==-250
[PASS] ┬°16.3 peek==500
[PASS] ┬°16.4 peek==-500
[PASS] ┬°17.1 pop-empty catchable
[PASS] ┬°17.2 peek-empty catchable
[PASS] ┬°17.3 push-full catchable
[PASS] ┬°18.1 empty exactly
[PASS] ┬°18.2 one-more pop throws
[PASS] ┬°19.1 peek==77
[PASS] ┬°19.2 peek==77 again
[PASS] ┬°19.3 !isEmpty
[PASS] ┬°19.4 empty after pop
[PASS] ┬°20.1 peek==-999999
[PASS] ┬°20.2 peek==999999

=== 183 / 183 tests passed ===

Tekan Enter buat nutup...
```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Di implementasi kode stack.cpp menggunakan mekanisme exception handling (throw) untuk menangani kondisi tidak valid sesuai di soal.

Penjelasan alur logikanya:

`init`: Mengeset pointer top ke nullptr. Karena stack diimplementasikan dengan array statis, `nullptr` adalah representasi logis bahwa stack benar-benar kosong.

`push`: Mengecek status `isFull`. Jika tidak penuh, pointer `top` dinaikkan ke alamat memori berikutnya, lalu diisi dengan nilai baru. Sesuai prinsip LIFO (Last In First Out), di mana data terakhir masuk selalu berada di posisi paling atas (alamat memori tertinggi saat itu).

`pop`: Mengecek status `isEmpty`. Jika tidak kosong, pointer `top` diturunkan ke alamat memori sebelumnya. Dengan menurunkan pointer, elemen teratas secara logis dihapus karena tidak lagi dapat diakses melalui fungsi `peek`, menjaga integritas urutan tumpukan.

`peek`: Mengambil nilai dari alamat yang ditunjuk oleh pointer `top` tanpa mengubah posisi pointer tersebut. Operasi ini memenuhi syarat akses data teratas tanpa memodifikasi struktur internal stack.

Analisis Kompleksitas Big-O

Fungsi	Waktu (Time)	Ruang (Space)	Penjelasan
init	O(1)	O(1)	Hanya melakukan satu operasi penugasan (assignment).
isEmpty	O(1)	O(1)	Hanya satu perbandingan pointer.
isFull	O(1)	O(1)	Membandingkan dua alamat memori secara langsung.
push	O(1)	O(1)	Operasi aritmatika pointer dan penulisan memori konstan.

pop	O(1)	O(1)	Operasi aritmatika pointer konstan.
peek	O(1)	O(1)	Pengambilan nilai melalui pointer dilakukan secara langsung.

Dan untuk mengujinya, di terminal vscode dimasukkan perintah sebagai berikut:

```
g++ -o test_stack tests/test_stack.cpp stack.cp
```

Tapi sebelum menjalankan perintah ini, di test_stack.cpp baris kode sebelum `return`, ditambahkan

```
std::cout << "\nTekan Enter buat nutup...";

std::cin.get();
```

Supaya ketika menjalankan hasil output kodenya yaitu test_stack.exe, bisa melihat hasilnya dan tidak langsung tertutup.

Untuk hasilnya bisa dilihat di B. Output Program

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• Ai adalah nilai bilangan pada input

• Simbol yang valid hanya:

‣bilangan bulat

‣operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 4 Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

```

```
bool isOperator(const string& s) {
    return (s == "+" || s == "-" || s == "*" || s == "/");
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Stack s;
    init(&s);

    for (int i = 0; i < n; ++i) {
        string simbol;
        cin >> simbol;

        if (isOperator(simbol)) {
            int operand2 = peek(&s); pop(&s);
            int operand1 = peek(&s); pop(&s);
            int hasil;

            if (simbol == "+") hasil = operand1 + operand2;
            else if (simbol == "-") hasil = operand1 - operand2;
            else if (simbol == "*") hasil = operand1 * operand2;
            else if (simbol == "/") {
                if (operand2 == 0) return 1;
                hasil = operand1 / operand2;
            }

            push(&s, hasil);
        }
        else {
            push(&s, stoi(simbol));
        }
    }

    cout << peek(&s) << endl;

    return 0;
}
```

Tabel 5 Source Code stack.cpp

```
#include "stack.h"
```

```
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Stack Overflow");
    }
    if (isEmpty(s)) {
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack Underflow");
    }
    if (s->top == &(s->data[0])) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack is Empty");
    }
    return *(s->top);
}
```

Tabel 6 Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

B. Output Program

```
D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob1.exe
3
2 3 +
5

D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob1.exe
9
5 1 2 + 4 * + 3 -
14

D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob1.exe
5
10 2 / 3 *
15
```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program ini terdiri dari **3 file utama**:

1. prob1.cpp → program utama
2. stack.cpp → implementasi stack

3. stack.h → deklarasi stack

1. **Program Utama prob1.cpp**

Dalam kode probe1.cpp ini berfungsi untuk:

- membaca input
- menentukan apakah simbol adalah angka atau operator
- melakukan perhitungan menggunakan stack

Alurnya sebagai berikut

1. **Membaca input**
    - Program membaca jumlah simbol (n)
    - Kemudian membaca simbol satu per satu
2. **Identifikasi simbol**
    - Jika simbol adalah operator (+, -, *, /) → diproses
    - Jika bukan → dianggap bilangan
3. **Jika simbol bilangan**
    - Nilai langsung dimasukkan ke dalam stack
4. **Jika simbol operator**
    - Mengambil dua nilai teratas dari stack
    - Nilai pertama yang diambil menjadi operand kedua
    - Nilai kedua menjadi operand pertama
    - Melakukan operasi sesuai operator
    - Hasilnya dimasukkan kembali ke stack
5. **Menampilkan hasil**
    - Setelah semua simbol diproses
    - Nilai terakhir di stack ditampilkan sebagai output

2. **Header stack.h**

Kode ini berisi definisi struktur dan deklarasi fungsi stack.

1. **Struktur Stack**

    - Menggunakan array untuk menyimpan data
    - Pointer `top` untuk menunjuk elemen teratas
2. **Kapasitas stack**
    - Maksimal 100 elemen
3. **Deklarasi fungsi**
    - `init` → inisialisasi stack
    - `isEmpty` → cek kosong
    - `isFull` → cek penuh
    - `push` → tambah data
    - `pop` → hapus data
    - `peek` → ambil data teratas

3. Implementasi Stack stack.cpp

File ini berisi implementasi dari fungsi-fungsi stack.

1. **init**
    - Mengosongkan stack dengan mengatur `top = nullptr`
2. **isEmpty**
    - Mengembalikan nilai true jika stack kosong
3. **isFull**
    - Mengecek apakah stack sudah mencapai kapasitas maksimum
4. **push**
    - Menambahkan elemen ke stack
    - Jika stack kosong → isi dari awal
    - Jika tidak → geser `top` ke atas
    - Menyimpan nilai baru di posisi `top`
5. **pop**
    - Menghapus elemen teratas
    - Jika hanya satu elemen → stack jadi kosong
    - Jika lebih → `top` digeser ke bawah
6. **peek**

- Mengambil nilai paling atas tanpa menghapusnya

Contoh input outputnya bisa dilihat di B. Output Program.

Untuk penjelasan alur prosesnya bisa dilihat dari tabel berikut:

Langkah	Simbol	Proses	Isi Stack
1	5	push	[5]
2	1	push	[5, 1]
3	2	push	[5, 1, 2]
4	+	1 + 2 = 3	[5, 3]
5	4	push	[5, 3, 4]
6	*	3 × 4 = 12	[5, 12]
7	+	5 + 12 = 17	[17]
8	3	push	[17, 3]
9	-	17 - 3 = 14	[14]

Penjelasan lanjutan Kode Program

stack.h

Di file ini, struktur `Stack` pakai konsep array statis + pointer **`top`**. `data[MAX]` jadi tempat nyimpen elemen, sementara `top` nunjuk ke elemen paling atas atau last inserted. Ini implementasi klasik stack berbasis array, tapi pakai pointer.

`MAX` jadi batas kapasitas stack, artinya ukuran fix dan nggak bisa grow. Ini bikin operasi lebih cepat dan sederhana.

Operasi yang disediain: `push`, `pop`, `peek`, plus helper (`isEmpty`, `isFull`, `init`).

Secara desain, header ini define struktur dan fungsi. File lain tinggal include tanpa perlu tau implementasi detail.

Stack.cpp

Di file ini, Implementasi di dimulai dari `init()` yang set `top = nullptr` → indikator stack kosong. Ini konsisten sama queue tadi: pakai `nullptr` sebagai state empty.

`isFull()` ngecek apakah `top` sudah di elemen terakhir array (`data[MAX-1]`).

`push()` punya dua kondisi: kalau kosong, `top` diarahkan ke `data[0]`; kalau tidak, `top++` (naik satu posisi). Setelah itu nilai dimasukkan ke `*top`. Ini implementasi langsung dari konsep stack karena elemen selalu masuk dari atas.

`pop()` kebalikannya: kalau cuma satu elemen (`top == data[0]`), maka stack direset jadi kosong (`nullptr`); kalau tidak, `top--`. Jadi elemen dibuang tanpa perlu dihapus dari array , cukup geser pointer.

`peek()` cuma return nilai di `top` tanpa modifikasi.

prob1.cpp

Program ini implementasi evaluasi ekspresi postfix pakai stack. Input berupa `n` token (angka atau operator), lalu diproses satu per satu. Fungsi `isOperator()` dipakai buat ngecek apakah token itu operator (`+ - * /`) atau operand.

Alur utamanya: kalau token adalah angka → langsung di-push ke stack. Kalau operator → ambil dua elemen teratas dari stack (`operand2` dulu, baru `operand1`), lalu lakukan operasi sesuai operatornya.

Urutan pengambilan operand itu: `operand1 op operand2`, bukan kebalik.

Hasil operasi disimpan di variabel `hasil`, lalu di-push lagi ke stack. Jadi stack selalu nyimpen state sementara dari evaluasi ekspresi sampai akhirnya tinggal satu nilai.

Di akhir, `peek(&s)` dipakai buat ambil hasil final. Algoritma ini berjalan dalam **O(n)**.

SOAL 3

A. Source Code

Tabel 7 Source Code test_queue.cpp

```
// tests/test_queue.cpp
//  Exhaustive  unit  tests  for  Queue  ADT  (queue.h  /
queue.cpp)
// exit 0 = all pass, exit 1 = any failure

#include "../queue.h"
#include <iostream>
#include <string>

static int g_passed = 0, g_failed = 0;

void test(const std::string& name, bool cond) {
    if (cond) { std::cout << "[PASS] " << name << "\n";
++g_passed; }
    else       { std::cout << "[FAIL] " << name << "\n";
++g_failed; }
}

template<typename Fn>
bool throws(Fn fn) {
    try { fn(); return false; }
    catch (...) { return true; }
}

int main() {
    std::cout << "=== Queue Unit Tests ===\n\n";

    // §1. Initialization
    Queue q;
    init(&q);
    test("§1.1 init: isEmpty",          isEmpty(&q));
    test("§1.2 init: !isFull",         !isFull(&q));
    test("§1.3  init:  front  throws",        throws([&]{
front(&q); }));
    test("§1.4  init:  back  throws",         throws([&]{
back(&q); }));
    test("§1.5  init:  dequeue  throws",      throws([&]{
dequeue(&q); }));

    // §2. Single enqueue/dequeue
    enqueue(&q, 42);
    test("§2.1 enqueue(42): !isEmpty",  !isEmpty(&q));
```

```
    test("§2.2 enqueue(42): !isFull",   !isFull(&q));
    test("§2.3 front==42",                  front(&q)  ==
42);
    test("§2.4 back==42",                  back(&q)   ==
42);
    dequeue(&q);
    test("§2.5 isEmpty after dequeue",   isEmpty(&q));
    test("§2.6  front  throws  after  deq",   throws([&]{
front(&q); }));
    test("§2.7  back  throws  after  deq",     throws([&]{
back(&q);  }));
    test("§2.8  dequeue  throws  after  deq",throws([&]{
dequeue(&q); }));

    // §3. FIFO ordering — 3 elements
    enqueue(&q, 10); enqueue(&q, 20); enqueue(&q, 30);
    test("§3.1 front==10", front(&q)==10);
    test("§3.2 back==30",  back(&q)==30);
    dequeue(&q);
    test("§3.3 front==20 after deq", front(&q)==20);
    test("§3.4 back==30 unchanged",  back(&q)==30);
    dequeue(&q);
    test("§3.5 front==30", front(&q)==30);
    test("§3.6 back==30",  back(&q)==30);
    dequeue(&q);
    test("§3.7 isEmpty after 3 dequeues", isEmpty(&q));

    // §4. FIFO — 10 elements sequential
    for (int i = 1; i <= 10; ++i) enqueue(&q, i * 100);
    for (int i = 1; i <= 10; ++i) {
        test("§4 FIFO front==" + std::to_string(i*100),
front(&q) == i*100);
        dequeue(&q);
    }
    test("§4.end empty", isEmpty(&q));

    // §5. front and back always correct
    enqueue(&q,  1);  enqueue(&q,  2);  enqueue(&q,  3);
enqueue(&q, 4); enqueue(&q, 5);
    test("§5.1 front==1", front(&q)==1);
    test("§5.2 back==5",  back(&q)==5);
    dequeue(&q);
    test("§5.3 front==2", front(&q)==2);
    test("§5.4 back==5",  back(&q)==5);
    dequeue(&q); dequeue(&q);
    test("§5.5 front==4", front(&q)==4);
    test("§5.6 back==5",  back(&q)==5);
```

```
    dequeue(&q);
    test("§5.7 front==5 == back==5", front(&q)==5 &&
back(&q)==5);
    dequeue(&q);
    test("§5.8 isEmpty", isEmpty(&q));

    // §6. Boundary values
    enqueue(&q, -1000); enqueue(&q, 0); enqueue(&q,
1000);
    test("§6.1 front==-1000", front(&q)==-1000);
    test("§6.2 back==1000",   back(&q)==1000);
    dequeue(&q);
    test("§6.3 front==0",     front(&q)==0);
    dequeue(&q);
    test("§6.4 front==1000 == back", front(&q)==1000 &&
back(&q)==1000);
    dequeue(&q);
    test("§6.5 empty", isEmpty(&q));

    // §7. Duplicate values
    enqueue(&q, 5); enqueue(&q, 5); enqueue(&q, 5);
    test("§7.1 front==5", front(&q)==5);
    test("§7.2 back==5",  back(&q)==5);
    dequeue(&q);
    test("§7.3 front==5 still", front(&q)==5);
    dequeue(&q); dequeue(&q);
    test("§7.4 empty after dups", isEmpty(&q));

    // §8. isFull at MAX capacity
    for (int i = 0; i < MAX; ++i) enqueue(&q, i);
    test("§8.1 isFull",             isFull(&q));
    test("§8.2 !isEmpty",          !isEmpty(&q));
    test("§8.3     enqueue-full      throws",throws([&]{
enqueue(&q, 999); }));
    test("§8.4 front==0",           front(&q) == 0);
    test("§8.5 back==MAX-1",       back(&q) == MAX - 1);

    // §9. Drain full, check FIFO order
    for (int i = 0; i < MAX; ++i) {
        test("§9  drain  front=="  +  std::to_string(i),
front(&q) == i);
        dequeue(&q);
    }
    test("§9.end empty after full drain", isEmpty(&q));

    // §10. Re-init resets
    enqueue(&q, 55); enqueue(&q, 66);
```

```
    init(&q);
    test("§10.1 isEmpty after re-init",
isEmpty(&q));
    test("§10.2 front throws after re-init",
throws([&]{ front(&q); }));
    test("§10.3 back throws after re-init",
throws([&]{ back(&q); }));
    test("§10.4 dequeue throws after re-init",
throws([&]{ dequeue(&q); }));
    enqueue(&q, 77);
    test("§10.5 enqueue after re-init ok",
front(&q)==77 && back(&q)==77);
    dequeue(&q);
    test("§10.6 empty after post-reinit pop",
isEmpty(&q));

    // §11. Multiple independent queues
    Queue q2;
    init(&q2);
    enqueue(&q, 100); enqueue(&q2, 200);
    test("§11.1 q front==100",  front(&q)==100);
    test("§11.2 q2 front==200", front(&q2)==200);
    enqueue(&q, 101); enqueue(&q2, 201);
    test("§11.3 q back==101",   back(&q)==101);
    test("§11.4 q2 back==201",  back(&q2)==201);
    dequeue(&q); dequeue(&q);
    dequeue(&q2); dequeue(&q2);
    test("§11.5 q empty",  isEmpty(&q));
    test("§11.6 q2 empty", isEmpty(&q2));

    // §12. isFull/isEmpty boundary dance
    for (int i = 0; i < MAX; ++i) enqueue(&q, i);
    test("§12.1 isFull",                isFull(&q));
    dequeue(&q);
    test("§12.2 !isFull after 1 deq",  !isFull(&q));
    enqueue(&q, 9999);
    test("§12.3 isFull after re-fill",  isFull(&q));
    test("§12.4 enqueue still throws", throws([&]{
enqueue(&q, 1); }));

    // §13. Drain full, refill negatives
    while (!isEmpty(&q)) dequeue(&q);
    test("§13.1 drained", isEmpty(&q));
    for (int i = 0; i < MAX; ++i) enqueue(&q, -i);
    test("§13.2 isFull",        isFull(&q));
    test("§13.3 front==0",      front(&q) == 0);
    test("§13.4 back==-(MAX-1)",back(&q) == -(MAX - 1));
```

```

    // §14. Negative values FIFO
    while (!isEmpty(&q)) dequeue(&q);
    enqueue(&q, -3); enqueue(&q, -2); enqueue(&q, -1);
    test("§14.1 front==-3", front(&q)==-3); dequeue(&q);
    test("§14.2 front==-2", front(&q)==-2); dequeue(&q);
    test("§14.3 front==-1", front(&q)==-1); dequeue(&q);

    // §15. Zero-only queue
    enqueue(&q, 0); enqueue(&q, 0); enqueue(&q, 0);
    test("§15.1     front==0     #1",     front(&q)==0);
dequeue(&q);
    test("§15.2     front==0     #2",     front(&q)==0);
dequeue(&q);
    test("§15.3     front==0     #3",     front(&q)==0);
dequeue(&q);
    test("§15.4 empty", isEmpty(&q));

    // §16. Interleaved enqueue/dequeue mix
    enqueue(&q, 1); enqueue(&q, 2);
    dequeue(&q);
    enqueue(&q, 3);
    dequeue(&q);
    enqueue(&q, 4); enqueue(&q, 5);
    test("§16.1 front==3", front(&q)==3);
    test("§16.2 back==5",  back(&q)==5);
    dequeue(&q); dequeue(&q); dequeue(&q);
    test("§16.3 empty after interleaved", isEmpty(&q));

    // §17. front does not modify queue
    enqueue(&q, 88);
    test("§17.1 front==88",       front(&q)==88);
    test("§17.2 front==88 again", front(&q)==88);
    test("§17.3 !isEmpty",       !isEmpty(&q));
    dequeue(&q);
    test("§17.4 empty after dequeue", isEmpty(&q));

    // §18. back does not modify queue
    enqueue(&q, 11); enqueue(&q, 22);
    test("§18.1 back==22",       back(&q)==22);
    test("§18.2 back==22 again", back(&q)==22);
    test("§18.3 front still 11", front(&q)==11);
    dequeue(&q); dequeue(&q);
    test("§18.4 empty", isEmpty(&q));

    // §19. Exception catchable
    bool e1=false, e2=false, e3=false, e4=false;
```

```
    try { dequeue(&q); } catch(...){ e1=true; }
    test("§19.1 dequeue-empty catchable", e1);
    try { front(&q);   } catch(...){ e2=true; }
    test("§19.2 front-empty catchable",   e2);
    try { back(&q);    } catch(...){ e3=true; }
    test("§19.3 back-empty catchable",    e3);
    for (int i=0; i<MAX; ++i) enqueue(&q, i);
    try { enqueue(&q,1); } catch(...){ e4=true; }
    test("§19.4 enqueue-full catchable",  e4);

    // §20. Exactly to empty then one more throws
    while (!isEmpty(&q)) dequeue(&q);
    enqueue(&q, 1); enqueue(&q, 2);
    dequeue(&q); dequeue(&q);
    test("§20.1 exact empty",            isEmpty(&q));
    test("§20.2  dequeue  throws  exactly",  throws([&]{
dequeue(&q); }));

    // Summary
    std::cout  <<  "\n===  "  <<  g_passed  <<  "  /  "  <<
(g_passed + g_failed)
              << " tests passed ===\n";
    std::cout << "\nTekan Enter buat nutup...";
    std::cin.get();
    return (g_failed == 0) ? 0 : 1;
}
```

Tabel 8 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;

    const int* next_rear;

    if (q->rear == &(q->data[MAX - 1])) {
```

```
        next_rear = &(q->data[0]);
    } else {
        next_rear = q->rear + 1;
    }

    return next_rear == q->front;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw    std::overflow_error("Queue    Overflow: Antrean penuh");
    }

    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        if (q->rear == &(q->data[MAX - 1])) {
            q->rear = &(q->data[0]);
        } else {
            q->rear++;
        }
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw   std::underflow_error("Queue   Underflow: Antrean kosong");
    }

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        if (q->front == &(q->data[MAX - 1])) {
            q->front = &(q->data[0]);
        } else {
            q->front++;
        }
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
```

```
        throw std::underflow_error("Queue is Empty");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue is Empty");
    }
    return *(q->rear);
}
```

Tabel 9 Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

B. Output Program

```
D:\Penyimpanan Utama\Docu

[PASS] ┬°10.1 isEmpty after re-init
[PASS] ┬°10.2 front throws after re-init
[PASS] ┬°10.3 back throws after re-init
[PASS] ┬°10.4 dequeue throws after re-init
[PASS] ┬°10.5 enqueue after re-init ok
[PASS] ┬°10.6 empty after post-reinit pop
[PASS] ┬°11.1 q front==100
[PASS] ┬°11.2 q2 front==200
[PASS] ┬°11.3 q back==101
[PASS] ┬°11.4 q2 back==201
[PASS] ┬°11.5 q empty
[PASS] ┬°11.6 q2 empty
[PASS] ┬°12.1 isFull
[PASS] ┬°12.2 !isFull after 1 deq
[PASS] ┬°12.3 isFull after re-fill
[PASS] ┬°12.4 enqueue still throws
[PASS] ┬°13.1 drained
[PASS] ┬°13.2 isFull
[PASS] ┬°13.3 front==0
[PASS] ┬°13.4 back==-(MAX-1)
[PASS] ┬°14.1 front==-3
[PASS] ┬°14.2 front==-2
[PASS] ┬°14.3 front==-1
[PASS] ┬°15.1 front==0 #1
[PASS] ┬°15.2 front==0 #2
[PASS] ┬°15.3 front==0 #3
[PASS] ┬°15.4 empty
[PASS] ┬°16.1 front==3
[PASS] ┬°16.2 back==5
[PASS] ┬°16.3 empty after interleaved
[PASS] ┬°17.1 front==88
[PASS] ┬°17.2 front==88 again
[PASS] ┬°17.3 !isEmpty
[PASS] ┬°17.4 empty after dequeue
[PASS] ┬°18.1 back==22
[PASS] ┬°18.2 back==22 again
[PASS] ┬°18.3 front still 11
[PASS] ┬°18.4 empty
[PASS] ┬°19.1 dequeue-empty catchable
[PASS] ┬°19.2 front-empty catchable
[PASS] ┬°19.3 back-empty catchable
[PASS] ┬°19.4 enqueue-full catchable
[PASS] ┬°20.1 exact empty
[PASS] ┬°20.2 dequeue throws exactly

=== 198 / 198 tests passed ===

Tekan Enter buat nutup...
```

Gambar 3. Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Implementasi di queue.cpp ini menggunakan konsep Circular Queue atau antrean melingkar untuk mengoptimalkan penggunaan memori array statis.

Alurnya sebagai berikut:

- Mekanisme Circular: Algoritma ini memastikan bahwa saat pointer `rear` atau `front` mencapai indeks terakhir (`MAX-1`), mereka akan diarahkan kembali ke alamat memori indeks ke-0 (`&data[0]`). Hal ini mencegah masalah *false overflow* pada queue berbasis array, di mana antrean dianggap penuh padahal ada ruang kosong di bagian depan akibat operasi `dequeue`.
- Aritmatika Pointer: Penggunaan `q->rear++` dan `q->front++` memungkinkan akses memori langsung tanpa perlu menghitung indeks array secara manual.
- Penanganan Error: Penggunaan `throw std::overflow_error` dan `underflow_error` memastikan program memberikan sinyal yang jelas saat terjadi kesalahan logika pada struktur data.

Analisis kompleksitas Big-O

Fungsi	Kompleksitas Waktu	Kompleksitas Ruang	Justifikasi
init	O(1)	O(1)	Inisialisasi pointer langsung ke null.
isEmpty	O(1)	O(1)	Hanya satu perbandingan pointer front.
isFull	O(1)	O(1)	Perbandingan posisi alamat memori antara next_rear dan front.
enqueue	O(1)	O(1)	Penulisan data langsung ke alamat memori yang ditunjuk rear.
dequeue	O(1)	O(1)	Hanya memindahkan alamat pointer front tanpa menggeser data array.

front/back	O(1)	O(1)	Akses langsung ke nilai yang ditunjuk oleh pointer.

Algoritma menjamin elemen yang pertama kali di-`enqueue` akan selalu berada di posisi yang ditunjuk `front`, dan akan menjadi yang pertama keluar saat `dequeue` dipanggil.

Penggunaan kondisi `if (q->front == q->rear)` pada fungsi `dequeue` memastikan bahwa saat elemen terakhir diambil, queue kembali ke status awal (`nullptr`) untuk menghindari akses memori ilegal.

Dan untuk mengujinya sama seperti di soal 1, di terminal vscode dimasukkan perintah sebagai berikut:

```
g++ -o test_queue tests/test_queue.cpp queue.cp
```

Tapi sebelum menjalankan perintah ini, di test_queue.cpp baris kode sebelum `return`, ditambahkan

```
std::cout << "\nTekan Enter buat nutup...";

std::cin.get();
```

Supaya ketika menjalankan hasil output kodenya yaitu test_queue.exe, bisa melihat hasilnya dan tidak langsung tertutup.

Untuk hasilnya bisa dilihat di B. Output Program

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 10 Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    int A[MAX];
    for (int i = 0; i < n; i++) {
        cin >> A[i];
```

```
    }

    Queue q;
    init(&q);

    int current_sum = 0;

    for (int i = 0; i < k; i++) {
        enqueue(&q, A[i]);
        current_sum += A[i];
    }
    cout << current_sum;

    for (int i = k; i < n; i++) {
        current_sum = current_sum - front(&q) + A[i];
        dequeue(&q);

        enqueue(&q, A[i]);

        cout << " " << current_sum;
    }

    cout << '\n';
    return 0;
}
```

Tabel 11 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;

    const int* next_rear = (q->rear == q->data + MAX - 1) ? q->data : q->rear + 1;
    return next_rear == q->front;
}
```

```

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw    std::overflow_error("Queue    Overflow: Antrean penuh");
    }

    if (isEmpty(q)) {
        q->front = q->data;
        q->rear = q->data;
    } else {
        q->rear = (q->rear == q->data + MAX - 1) ? q->data : q->rear + 1;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw   std::underflow_error("Queue   Underflow: Antrean kosong");
    }

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front = (q->front == q->data + MAX - 1) ? q->data : q->front + 1;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue is Empty");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue is Empty");
    }
    return *(q->rear);
}
```

Tabel 12 Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

B. Output Program

```
D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob2.exe
5 3
1 2 3 4 5
6 9 12

D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob2.exe
5 5
1 2 3 4 5
15

D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PRAKTIKUM\module-2-stack-and-queue-RyuuZev>prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Program ini digunakan untuk menghitung jumlah dari setiap kelompok elemen berurutan sebanyak k elemen dalam sebuah array.

Metode yang digunakan adalah sliding window atau jendela geser, yaitu:

- ambil k elemen pertama → hitung jumlah
- geser satu langkah ke kanan → update jumlah
- ulang sampai habis

Di sini ada 3 file utama yaitu, prob2.cpp queue.cpp queue.h

1. Program Utama prob2.cpp

Program utama berfungsi untuk:

- membaca input
- mengatur proses perhitungan sliding window
- menampilkan hasil

Alur nya sebagai berikut:

1. Membaca input
    - membaca nilai `n` (jumlah elemen) dan `k` (ukuran window)
    - membaca seluruh elemen array
2. Inisialisasi queue
    - queue digunakan untuk menyimpan elemen dalam window aktif
3. Menghitung window pertama
    - memasukkan k elemen pertama ke dalam queue
    - menghitung jumlahnya dan disimpan di `current_sum`
    - hasil langsung ditampilkan
4. Proses sliding window
    - untuk setiap elemen berikutnya:
        - mengambil elemen paling depan (yang keluar dari window)
        - menghapusnya dari queue
        - memasukkan elemen baru
        - memperbarui jumlah dengan rumus:

          ```
          current_sum = current_sum - elemen_keluar + elemen_baru
          ```

    - hasil setiap window ditampilkan

5. Menampilkan output
    - seluruh hasil dicetak dalam satu baris dipisahkan spasi

2. Header queue.h

Kode ini berisi definisi struktur queue dan deklarasi fungsi.

1. Struktur Queue
    - menggunakan array untuk menyimpan data
    - pointer `front` → menunjuk elemen depan
    - pointer `rear` → menunjuk elemen belakang
2. Kapasitas
    - maksimal 100 elemen
3. Fungsi yang tersedia
    - `init` → inisialisasi queue
    - `isEmpty` → mengecek apakah kosong
    - `isFull` → mengecek apakah penuh
    - `enqueue` → menambahkan elemen
    - `dequeue` → menghapus elemen depan
    - `front` → mengambil elemen depan
    - `back` → mengambil elemen belakang

3. Impelmentasi queue.cpp

Queue diimplementasikan menggunakan circular queue (antrian melingkar).

1. init
    - mengatur queue menjadi kosong (`front` dan `rear = nullptr`)
2. isEmpty
    - mengembalikan nilai true jika queue kosong
3. isFull
    - queue penuh jika posisi berikutnya dari `rear` adalah `front`
4. enqueue

    - menambahkan elemen ke belakang
    - jika sudah di ujung array → kembali ke awal (circular)
5. dequeue
    - menghapus elemen dari depan
    - jika hanya satu elemen → queue kembali kosong
6. front
    - mengambil nilai elemen paling depan
7. back
    - mengambil nilai elemen paling belakang

Contoh input outputnya bisa dilihat di B. Output Program.

Untuk penjelasan alur prosesnya bisa dilihat dari tabel berikut:

Window	**Elemen**	**Jumlah**
1	[4, -1]	3
2	[-1, 2]	1
3	[2, -3]	-1
4	[-3, 5]	2
5	[5, 1]	6

Output

```
3 1 -1 2 6
```

Penjelasan lanjutan Kode Program

Stack.h

Struktur `Queue` di pakai pendekatan array statis + pointer, bukan indeks. `data[MAX]` jadi storage, sementara `front` dan `rear` adalah pointer yang langsung nunjuk ke elemen dalam array. Ini beda dari implementasi umum yang pakai integer index, di sini lebih low-level dan fleksibel.

Konsep pentingnya adalah circular queue, tapi nggak eksplisit di header. Indikasinya ada di penggunaan pointer yang nanti bisa muter ke awal array. Jadi queue ini bukan linear yang bakal cepat penuh, tapi reusable space.

`MAX` jadi batas kapasitas queue. Karena array statis, ukuran queue fix di compile-time. Ini simpel dan cepat, tapi kurang fleksibel dibanding dynamic allocation (misalnya pakai linked list).

Semua operasi queue (init, enqueue, dequeue, dll.) cuma dideklarasi di sini. Header ini jadi kontrak yang dipakai file lain tanpa peduli detail implementasinya.

Queue.cpp

Di file ini, `init()` set `front` dan `rear` ke `nullptr`, yang jadi indikator queue kosong. Ini desain yang clean karena cukup satu kondisi untuk cek empty (`front == nullptr`).

`isFull()` adalah bagian paling krusial karena hitung posisi next dari `rear`, kalau sudah di ujung array maka wrap ke `data` (awal array). Kalau posisi next ini sama dengan `front`, berarti queue penuh.

`enqueue()` punya dua skenario: kalau kosong, `front` dan `rear` di-set ke awal array, kalau tidak, `rear` digeser maju atau wrap. Setelah itu value dimasukkan ke posisi `rear`. Ini memastikan elemen selalu ditambah di belakang (FIFO).

`dequeue()`: kalau cuma ada satu elemen (`front == rear`), queue di-reset jadi kosong. Kalau tidak, `front` digeser maju atau wrap. Ini menjaga pointer tetap valid tanpa perlu shifting data.

prob2.cpp

Program di sini baca input `n` (jumlah elemen) dan `k` (ukuran window), lalu simpan array `A`. Queue dipakai sebagai struktur bantu, bukan sekadar latihan implementasi.

Bagian awal: isi queue dengan `k` elemen pertama sambil hitung `current_sum`. Ini jadi baseline untuk window pertama. Output langsung dicetak tanpa spasi di depan.

Loop utama mulai dari index `k`: tiap iterasi, elemen paling depan di queue dihapus karena keluar dari window, lalu elemen baru ditambahkan. `current_sum` diupdate dengan cara incremental: `- front + A[i]`.

Queue di sini berfungsi sebagai representasi window aktif. Tanpa queue, mesti harus tracking indeks manual atau shifting array, lebih ribet dan rawan bug.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-RyuuZev